DealFlow360 — Frontend Explained (Judge Reference)

> ***Stack***: *React 18 · Vite 6 · Tailwind CSS 3 · Zustand · Socket.io-client · Axios · Recharts* ***URL***: *http://localhost:5173* ***Source root***: `frontend/src/` ***Total pages***: *19* · ***Total components***: *13* · ***Total lines***: *~12,000*

Table of Contents

---

1. Architecture Overview

```
frontend/
■■■ src/
■   ■■■ main.jsx              ← React DOM root mount
■   ■■■ App.jsx               ← BrowserRouter + all Routes + Guard
■   ■■■ index.css             ← Tailwind directives + custom CSS tokens
■   ■■■ api/
■   ■   ■■■ client.js         ← Axios instance + request/response interceptors
■   ■   ■■■ index.js          ← All API function exports
■   ■■■ store/
■   ■   ■■■ authStore.js      ← Zustand: user, accessToken
■   ■   ■■■ dealStore.js      ← Zustand: pipeline deals state
■   ■■■ hooks/
■   ■   ■■■ useSocket.js      ← Socket.io-client hook
■   ■■■ components/
■   ■   ■■■ AppInitializer.jsx ← Token refresh on page load
■   ■   ■■■ layout/           ← AppLayout, Sidebar, Navbar
■   ■   ■■■ charts/           ← PipelineChart, ValuationChart
■   ■   ■■■ forms/            ← FileDropzone, NewDealModal
■   ■   ■■■ ui/               ← Reusable UI primitives
■   ■■■ pages/
■   ■   ■■■ auth/             ← Login, Signup
■   ■   ■■■ dashboard/        ← Dashboard
■   ■   ■■■ workspace/        ← Quotations, Builder, Kanban, Approvals, Fulfillment, Subscriptions, Invoices
■   ■   ■■■ backend/          ← Products, PriceLists, DiscountTiers, Warehouses, Plans, UpsellRules, Users
■   ■   ■■■ portal/           ← CustomerPortal, PortalLogin
■   ■■■ utils/
■       ■■■ formatters.js     ← formatCurrency, formatDate, STAGES, PRIORITY_COLORS
```

Data Flow Diagram

```
User Action
    ■
    ▼
React Component  ■■useState/useMemo■■■  Local UI State
    ■
    ■■■■ Zustand store (authStore / dealStore) ←■■■■ persisted in localStorage
    ■
    ■■■■ Axios API call  ■■■ Express backend (port 5000) ■■■ PostgreSQL
    ■         ■■■ interceptor auto-attaches Bearer token
    ■         ■■■ interceptor auto-refreshes on 401
    ■
    ■■■■ Socket.io  ■■■ server event ■■■ broadcast to all clients in room
              ■■■ real-time: risk scores, approvals, portal messages
```

2. Entry Points

`main.jsx`

```
import React from &#x27;react&#x27;;
import ReactDOM from &#x27;react-dom/client&#x27;;
import &#x27;./index.css&#x27;;
import App from &#x27;./App&#x27;;

ReactDOM.createRoot(document.getElementById(&#x27;root&#x27;)).render(
  &lt;React.StrictMode&gt;
    &lt;App /&gt;
  &lt;/React.StrictMode&gt;
);
```

Standard React 18 `createRoot` mount. StrictMode double-renders in dev to catch side effects.

`App.jsx` — Master Router (126 lines)

```
function Guard({ children, roles }) {
  const { user } = useAuthStore();
  if (!user) return &lt;Navigate to=&quot;/login&quot; replace /&gt;;
  if (roles &amp;&amp; !roles.includes(user.role))
    return &lt;Navigate to=&quot;/dashboard&quot; replace /&gt;;
  return children;
}
```

Guard is a simple HOC (Higher Order Component):

- If no user in Zustand → redirect to **`/login`**
- If user's role is not in the allowed ___CODESPAN_0_ **array** → **redirect to** _CODESPAN_1___
- Otherwise render children

Every protected route is wrapped: **`<Guard roles={['ADMIN']}><UsersPage /></Guard>`**

Customer Portal is intentionally **outside** the Guard — it uses a magic token URL, no login required.

`AppInitializer.jsx` — Token Hydration (46 lines)

```
useEffect(() =&gt; {
  const init = async () =&gt; {
    if (user) {
      const res = await fetch(&#x27;http://localhost:5000/api/auth/refresh&#x27;, {
        method: &#x27;POST&#x27;, credentials: &#x27;include&#x27;
      });
      if (res.ok) {
        const data = await res.json();
        setToken(data.accessToken);   // inject into Axios
        setAuth(user, data.accessToken);
      } else { logout(); }
    }
    setReady(true);
  };
  init();
}, []);
```

Why this exists: JWT access tokens expire in 15 minutes. On page refresh, Zustand re-hydrates ___CODESPAN_0_ **from localStorage (via** _CODESPAN_1_ **middleware), but the old access token is stale.** _CODESPAN_2_ **silently calls** _CODESPAN_3___ using the httpOnly refresh cookie to get a fresh access token — transparent to the user.

If refresh fails → force logout. Shows animated loading screen during this check.

3. Auth System

`Login.jsx` (307 lines)

Key features:

- Demo account cards for all 5 roles — clicking pre-fills the form
- Client-side validation (`emailRegex`, password min 8 chars)
- On success: stores ___CODESPAN_0_ + _CODESPAN_1_ **in Zustand, calls** _CODESPAN_2___ to inject into Axios

```
const demoAccounts = [
  { role: &#x27;ADMIN&#x27;,        email: &#x27;admin@dealflow.com&#x27;,   password: &#x27;Admin@123&#x27; },
  { role: &#x27;SALES_REP&#x27;,   email: &#x27;priya@dealflow.com&#x27;,   password: &#x27;Rep@123&#x27; },
  { role: &#x27;SALES_MANAGER&#x27;, email: &#x27;manager@dealflow.com&#x27;, password: &#x27;Manager@123&#x27; },
  { role: &#x27;FINANCE&#x27;,     email: &#x27;finance@dealflow.com&#x27;, password: &#x27;Finance@123&#x27; },
  { role: &#x27;CUSTOMER&#x27;,    email: &#x27;buyer@acme.com&#x27;,        password: &#x27;Customer@123&#x27; },
];
```

```
const res = await authAPI.login({ email, password });
setAuth(res.user, res.accessToken);  // into Zustand
setToken(res.accessToken);           // into Axios module-level variable
navigate(&#x27;/dashboard&#x27;);
```

Magic Link flow: separate input — sends email with a token URL `/portal/:token`. Used by CustomerPortal.

`Signup.jsx` (257 lines)

- Role selection: SALES_REP, SALES_MANAGER, FINANCE, ADMIN, CUSTOMER
- Full client-side validation: name, email, password strength, confirm password
- Calls `POST /api/auth/signup` → auto-login on success

4. State Management

`authStore.js` (14 lines) — Zustand

```
export const useAuthStore = create(persist(
  (set) =&gt; ({
    user: null,
    accessToken: null,
    setAuth: (user, accessToken) =&gt; set({ user, accessToken }),
    updateToken: (accessToken) =&gt; set({ accessToken }),
    logout: () =&gt; set({ user: null, accessToken: null }),
  }),
  { name: &#x27;dealflow-auth&#x27; }  // persists to localStorage key &#x27;dealflow-auth&#x27;
));
```

Why Zustand over Redux? Zustand has zero boilerplate — no actions, reducers, or dispatchers. The ___CODESPAN_0_ **middleware automatically serializes to localStorage so state survives page refreshes.** _CODESPAN_1___ (used in Axios interceptor) allows reading state outside React components.

Shape of `user` object:

```
{
  &quot;id&quot;: &quot;uuid&quot;,
  &quot;name&quot;: &quot;Admin User&quot;,
  &quot;email&quot;: &quot;admin@dealflow.com&quot;,
  &quot;role&quot;: &quot;ADMIN&quot;,
  &quot;tier&quot;: &quot;GOLD&quot;,
  &quot;isActive&quot;: true
}
```

`dealStore.js` — Zustand (Pipeline State)

```
export const useDealStore = create((set) =&gt; ({
  deals: [],
  isNewDealModalOpen: false,
  addOrUpdateDeal: (deal) =&gt; set((state) =&gt; ({ ... })),
  removeDeal: (id) =&gt; set((state) =&gt; ({ deals: state.deals.filter(d =&gt; d.id !== id) })),
  setIsNewDealModalOpen: (v) =&gt; set({ isNewDealModalOpen: v }),
}));
```

Used by ___CODESPAN_0_, _CODESPAN_1_ **hook,** _CODESPAN_2___ (opens modal).

5. API Layer

`client.js` — Axios (43 lines)

```
const api = axios.create({
  baseURL: &#x27;http://localhost:5000/api&#x27;,
  withCredentials: true,   // send cookies (for refresh token)
});

let _token = null;
export const setToken = (t) =&gt; { _token = t; };

// REQUEST interceptor: attach Bearer token to every call
api.interceptors.request.use((config) =&gt; {
  const token = _token || useAuthStore.getState().accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// RESPONSE interceptor: auto-refresh on 401
api.interceptors.response.use(
  (res) =&gt; res.data,    // unwrap .data from every response automatically
  async (error) =&gt; {
    if (error.response?.status === 401 &amp;&amp; !original._retry) {
      // silently get new access token using refresh cookie
      const res = await axios.post(&#x27;/api/auth/refresh&#x27;, {}, { withCredentials: true });
      _token = res.data.accessToken;
      useAuthStore.getState().updateToken(_token);
      return api(original);  // retry original request
    }
    return Promise.reject(error.response?.data || error);
  }
);
```

Key design decisions:

- ___CODESPAN_0_ **unwrap means every API call returns the data directly, no** _CODESPAN_1___ nesting
- **`_token`** is a module-level variable (not React state) so it's accessible from interceptors synchronously
- **`_retry`** flag prevents infinite loop on repeated 401s

`api/index.js` — All Endpoints

```
export const authAPI = {
  login: (body) =&gt; api.post(&#x27;/auth/login&#x27;, body),
  signup: (body) =&gt; api.post(&#x27;/auth/signup&#x27;, body),
  me: () =&gt; api.get(&#x27;/auth/me&#x27;),
  logout: () =&gt; api.post(&#x27;/auth/logout&#x27;),
  getUsers: () =&gt; api.get(&#x27;/auth/users&#x27;),
  ...
};

export const quotationsAPI = {
  list: (params) =&gt; api.get(&#x27;/quotations&#x27;, { params }),
  get: (id) =&gt; api.get(`/quotations/${id}`),
  create: (body) =&gt; api.post(&#x27;/quotations&#x27;, body),
  update: (id, body) =&gt; api.put(`/quotations/${id}`, body),
  submit: (id) =&gt; api.put(`/quotations/${id}/submit`),
  decision: (id, body) =&gt; api.put(`/quotations/${id}/decision`, body),
  computeRisk: (body) =&gt; api.post(&#x27;/quotations/compute-risk&#x27;, body),
  sendToCustomer: (id) =&gt; api.put(`/quotations/${id}/send`),
  getPdf: (id) =&gt; api.get(`/quotations/${id}/pdf`, { responseType: &#x27;blob&#x27; }),
  ...
};
// + productsAPI, fulfillmentAPI, subscriptionsAPI, invoicesAPI,
//   negotiationsAPI, dashboardAPI, notificationsAPI, usersAPI
```

6. Real-time Layer

`useSocket.js` (70 lines)

```
export const useSocket = () =&gt; {
  const socketRef = useRef(null);
  const [isConnected, setIsConnected] = useState(false);

  useEffect(() =&gt; {
    const socket = io(&#x27;http://localhost:5000&#x27;, {
      withCredentials: true,
      transports: [&#x27;websocket&#x27;, &#x27;polling&#x27;],
    });

    socket.on(&#x27;deal:created&#x27;, (deal) =&gt; {
      addOrUpdateDeal(deal);
      toast.success(`New Deal: ${deal.title}`, { icon: &#x27;■&#x27; });
    });

    socket.on(&#x27;deal:updated&#x27;, (deal) =&gt; addOrUpdateDeal(deal));
    socket.on(&#x27;deal:deleted&#x27;, ({ id }) =&gt; removeDeal(id));

    return () =&gt; socket.disconnect();  // cleanup on unmount
  }, []);
};
```

Socket rooms used across the app:

Room	Joined by	Events
`dashboard`	Dashboard.jsx	`dashboard:update`
`approvers`	ApprovalQueue.jsx	`approval:new`
`portal_{token}`	CustomerPortal.jsx	___CODESPAN_0_, _CODESPAN_1___
`quotation_{id}`	QuotationBuilder.jsx	`risk-result` (live risk scoring)
`workspace_{id}`	various	`deal:activity`

Live Risk Scoring (in QuotationBuilder):

```
socket.emit(&#x27;compute-risk-live&#x27;, { lines, customerId, ... });
socket.on(&#x27;risk-result&#x27;, (result) =&gt; {
  setRiskResult(result);  // { score, level, breakdown }
});
```

The server runs `blendedRiskEngine.js` and emits back the score within milliseconds — the UI updates the risk LED and approval path in real time as the user types.

---

7. Layout System

`AppLayout.jsx`

```
&lt;div className=&quot;flex h-screen overflow-hidden bg-slate-950&quot;&gt;
  &lt;AppSidebar /&gt;           {/* left nav: 256px wide */}
  &lt;div className=&quot;flex-1 flex flex-col overflow-hidden&quot;&gt;
    &lt;Navbar /&gt;              {/* top bar: 64px high */}
    &lt;main className=&quot;flex-1 overflow-y-auto p-6&quot;&gt;
      &lt;Outlet /&gt;            {/* react-router child page renders here */}
    &lt;/main&gt;
  &lt;/div&gt;
&lt;/div&gt;
```

___CODESPAN_0_ **is from React Router — the parent route renders** _CODESPAN_1_ **and child routes render inside the** _CODESPAN_2_**. This means the Sidebar and Navbar are never re-mounted on navigation — only the** _CODESPAN_3___ content swaps.

`Sidebar.jsx` (90 lines)

- Brand logo with gradient
- NavLink-based navigation with `isActive` styling
- Active link: `bg-brand-600/10 text-brand-400 border border-brand-500/20`
- "New Deal Mandate" button triggers `dealStore.setIsNewDealModalOpen(true)`

`Navbar.jsx`

- Shows logged-in user name + role badge

- `NotificationDropdown` component — bell icon with unread count badge
- Logout button → calls ___CODESPAN_0_ + _CODESPAN_1_ + _CODESPAN_2___

---

8. Dashboard Page

File: ___CODESPAN_0_ **— 621 lines Route:** _CODESPAN_1___ — accessible to all authenticated roles

State Structure

```
const [data, setData] = useState({
  kpis: {
    totalQuotations: 0,    confirmedDeals: 0,
    confirmedValue: 0,     pendingApprovals: 0,
    totalRevenue: 0,       draftQuotations: 0,
    rejectedQuotations: 0, activeSubscriptions: 0,
    avgDealSize: 0,
  },
  stalledDeals: [],
  discountAnomalies: [],
  expiringQuotations: [],
  pipelineChart: [],
  revenueTrend: [],
  topReps: [],
  reps: [],
});
```

Filters

```
const [period, setPeriod] = useState(&#x27;month&#x27;);   // today | week | month | custom
const [selectedRep, setSelectedRep] = useState(&#x27;&#x27;);
```

When filter changes → ___CODESPAN_0_ **re-runs** → **calls** _CODESPAN_1___

Charts Used (Recharts)

```
// Revenue trend — AreaChart
&lt;AreaChart data={data.revenueTrend}&gt;
  &lt;Area type=&quot;monotone&quot; dataKey=&quot;revenue&quot; stroke=&quot;#3b82f6&quot; fill=&quot;url(#gradient)&quot; /&gt;
&lt;/AreaChart&gt;

// Pipeline by status — BarChart
&lt;BarChart data={data.pipelineChart}&gt;
  &lt;Bar dataKey=&quot;count&quot; fill=&quot;#8b5cf6&quot; /&gt;
&lt;/BarChart&gt;
```

AI-style Anomaly Cards

Dashboard shows **Discount Anomalies** — quotations where discount > threshold:

```
{data.discountAnomalies.map(q =&gt; (
  &lt;div key={q.id} className=&quot;border border-amber-500/30 bg-amber-500/5 rounded-lg p-3&quot;&gt;
    &lt;div&gt;{q.quotationNumber} — {q.customerName}&lt;/div&gt;
    &lt;div className=&quot;text-amber-400&quot;&gt;Discount: {q.maxDiscount}% (avg)&lt;/div&gt;
  &lt;/div&gt;
))}
```

Socket.io in Dashboard

```
useEffect(() =&gt; {
  const socket = io(&#x27;http://localhost:5000&#x27;, { withCredentials: true });
  socket.emit(&#x27;join_dashboard&#x27;);
  socket.on(&#x27;dashboard:update&#x27;, () =&gt; fetchMetrics());  // auto-refresh on any deal change
  return () =&gt; socket.disconnect();
}, []);
```

---

9. Quotation Builder (CPQ)

File: ___CODESPAN_0_ **— 760 lines Route:** _CODESPAN_1_ **(create) or** _CODESPAN_2___ (edit/view)

This is the most complex page. It is the **Configure, Price, Quote** engine.

Data Model (line items)

```
const defaultLine = (product = null) =&gt; ({
  _id: genId(),               // temporary local ID (&#x27;tmp-abc123&#x27;)
  product_id: product?.id || &#x27;&#x27;,
  line_type: &#x27;ONE_TIME&#x27;,      // ONE_TIME | RECURRING | SERVICE
  quantity: 1,
  unit_price: product?.base_price || 0,
  cost_price: product?.cost_price || 0,
  discount: 0,                // percentage
  tax: product?.tax || 18,    // percentage (GST)
  notes: &#x27;&#x27;,
});
```

ProductPicker Sub-component

```
function ProductPicker({ products, onSelect, onClose }) {
  const [search, setSearch] = useState(&#x27;&#x27;);
  const filtered = products.filter(p =&gt;
    `${p.name} ${p.sku}`.toLowerCase().includes(search.toLowerCase())
  );
  // Renders modal overlay with search input + product cards
}
```

Opens as a modal overlay, auto-focuses the search input. Selecting a product calls `defaultLine(product)` and appends to lines array.

Live Price Calculation

Every time a line changes, `useMemo` recomputes totals:

```
const totals = useMemo(() =&gt; {
  return lines.reduce((acc, line) =&gt; {
    const effectivePrice = line.unit_price * (1 - line.discount / 100);
    const lineTotal = effectivePrice * line.quantity * (1 + line.tax / 100);
    acc.subtotal += effectivePrice * line.quantity;
    acc.taxAmount += effectivePrice * line.quantity * (line.tax / 100);
    acc.total += lineTotal;
    return acc;
  }, { subtotal: 0, taxAmount: 0, total: 0 });
}, [lines]);
```

Upsell Suggestions

```
const fetchUpsellSuggestions = useCallback(async () =&gt; {
  const productIds = lines.map(l =&gt; l.product_id).filter(Boolean);
  const res = await productsAPI.getUpsellSuggestions({ product_ids: productIds });
  setUpsellSuggestions(res.suggestions || []);
}, [lines]);
```

Called whenever lines change. Backend matches product IDs against `UpsellRule` table.

Submit → Approval Flow

```
DRAFT  →  [Submit]  →  PENDING_MANAGER  →  [Approve/Reject]
                                        →  PENDING_FINANCE  →  [Approve/Reject]
                                                            →  APPROVED
                                                               [Send to Customer]  →  SENT_TO_CUSTOMER
```

Quotation Status Machine

```
DRAFT → PENDING_MANAGER → PENDING_FINANCE → APPROVED → SENT_TO_CUSTOMER
     ■  REJECTED (at any approval stage)
                                        → UNDER_NEGOTIATION (customer negotiates)
                                        → CONFIRMED (customer confirms)
                                        → CANCELLED
```

---

10. Quotations List & Pipeline Kanban

`QuotationsList.jsx` (773 lines)

- Tab-based filter: All | Draft | Pending | Approved | Rejected | Sent | Confirmed
- Grid/List view toggle
- Search by quotation number or customer name
- Each card shows: status badge, customer, value, risk score, expiry
- Clicking a card → navigate to `/quotations/:id`

`PipelineKanban.jsx` (488 lines)

- **react-beautiful-dnd** library for drag-and-drop columns
- Columns based on ___CODESPAN_0_ **from** _CODESPAN_1___: LEAD → QUALIFICATION → DUE_DILIGENCE → NEGOTIATION → CLOSED_WON → CLOSED_LOST
- Dragging a card between columns updates the deal stage via API

```
const onDragEnd = async (result) =&gt; {
  const { destination, draggableId } = result;
  if (!destination) return;
  await quotationsAPI.updateStage(draggableId, destination.droppableId);
};
```

---

11. Approval Queue

File: ___CODESPAN_0_ **— 630 lines Route:** _CODESPAN_1___ — restricted to SALES_MANAGER, FINANCE, ADMIN

Features

- Lists all quotations with ___CODESPAN_0_ **or** _CODESPAN_1___ status
- **Batch Approve/Reject** — checkboxes + single action for multiple items

```
const handleBatchDecision = async (action) =&gt; {
  await quotationsAPI.batchDecision({
    ids: selectedIds,      // array of quotation IDs
    action,                // &#x27;approve&#x27; or &#x27;reject&#x27;
    comment,
  });
  fetchQueue();
};
```

- Individual approve/reject with comment modal
- Real-time update: ___CODESPAN_0_ → **receives** _CODESPAN_1___ events from backend

---

12. Fulfillment Page

File: ___CODESPAN_0_ **— 1,227 lines (largest page) Route:** _CODESPAN_1___

Warehouse Split Logic

When a quotation is approved, fulfillment checks warehouse stock per product. If one warehouse doesn't have enough stock, it suggests a **split shipment**:

```
const [splitPlan, setSplitPlan] = useState([]);

// Backend returns:
// [{ warehouseId, warehouseName, items: [{ productId, qty }] }]

const loadSplitPlan = async (quotationId) =&gt; {
  const plan = await fulfillmentAPI.getSplitPlan(quotationId);
  setSplitPlan(plan.splits);
};
```

- Shows a visual split allocation table per warehouse
- "Accept Split" button confirms the allocation plan

Warehouse Stock Grid

```
{warehouses.map(w =&gt; (
  &lt;div key={w.id} className=&quot;bg-slate-900 rounded-xl p-4&quot;&gt;
    &lt;h3&gt;{w.name} — {w.location}&lt;/h3&gt;
    {w.stock.map(s =&gt; (
      &lt;div key={s.productId}&gt;
        {s.product.name}: {s.quantity} units
        &lt;span className={s.quantity &lt; 10 ? &#x27;text-red-400&#x27; : &#x27;text-green-400&#x27;}&gt;
          {s.quantity &lt; 10 ? &#x27;■ Low Stock&#x27; : &#x27;✓ In Stock&#x27;}
        &lt;/span&gt;
      &lt;/div&gt;
    ))}
  &lt;/div&gt;
))}
```

13. Subscriptions & Invoices

`Subscriptions.jsx` (981 lines)

- Lists all active subscriptions (linked to confirmed quotations)
- Shows: plan name, customer, billing cycle, next renewal date, MRR
- Cancel subscription → calls `PUT /api/subscriptions/:id/cancel`
- Subscription Plans management (inline tab): create/edit recurring billing plans

`Invoices.jsx` (924 lines)

- Invoice CRUD — list, create, mark as paid
- **PDF Download**: calls ___CODESPAN_0_ **with** _CODESPAN_1___

```
const downloadPDF = async (id) =&gt; {
  const blob = await invoicesAPI.getPdf(id);  // returns Blob
  const url = URL.createObjectURL(blob);
  const a = document.createElement(&#x27;a&#x27;);
  a.href = url;
  a.download = `invoice-${id}.pdf`;
  a.click();
  URL.revokeObjectURL(url);
};
```

---

14. Customer Portal

File: ___CODESPAN_0_ **— 944 lines Route:** _CODESPAN_1___ — **NO auth guard** — public URL

This is the external-facing B2B negotiation interface for customers.

Token-based Access

```
const { token } = useParams();

useEffect(() =&gt; {
  const loadQuotation = async () =&gt; {
    const res = await quotationsAPI.getByPortalToken(token);
    // GET /api/quotations/portal/:token
    setQuotation(res.quotation);
  };
  loadQuotation();
}, [token]);
```

The ___CODESPAN_0_ **is a UUID stored on the** _CODESPAN_1___ model. When sales rep clicks "Send to Customer", the backend generates/sets this token and emails the URL.

QR Code

```
import { QRCodeSVG } from &#x27;qrcode.react&#x27;;

&lt;QRCodeSVG
  value={`http://localhost:5173/portal/${token}`}
  size={96}
  bgColor=&quot;transparent&quot;
  fgColor=&quot;#3b82f6&quot;
/&gt;
```

Customer can scan QR on mobile to open portal on their phone.

Live Negotiation Room

```
const socket = io(&#x27;http://localhost:5000&#x27;, { withCredentials: true });
socket.emit(&#x27;join_portal&#x27;, { token });

// Customer sends counteroffer message
socket.on(&#x27;negotiation:message&#x27;, (msg) =&gt; {
  setMessages(prev =&gt; [...prev, msg]);
});

const sendMessage = async () =&gt; {
  await negotiationsAPI.negotiate(quotation.id, {
    message: messageText,
    proposedPrice: counterPrice,
  });
};
```

Sales team sees the message in their `quotation_{id}` room in real-time.

Confirm Deal

```
const confirmDeal = async () =&gt; {
  await negotiationsAPI.confirmPortal(negotiation.id);
  // Quotation status → CONFIRMED
  toast.success(&#x27;Deal Confirmed! ■&#x27;);
};
```

Time-to-Expiry Countdown

```
const getRelativeTime = (dateStr) =&gt; {
  const diffSec = Math.floor((new Date() - new Date(dateStr)) / 1000);
  if (diffSec &lt; 60) return &#x27;Just now&#x27;;
  // ... minutes, hours, days
};
```

---

15. Backend Config Pages

These are admin-only pages for configuring the sales engine. All follow the same pattern:

`Fetch data from API → render table/cards → CRUD modals → re-fetch`

`Products.jsx` (745 lines)

- Product catalog with image upload via FastAPI / Multer-compatible endpoint
- Image preview using `URL.createObjectURL(file)` before upload
- Category filter, search, sort
- Product variant management (size, color, etc.)
- Stock tracking per warehouse

`PriceLists.jsx` (249 lines)

- Customer-tier based pricing (BRONZE, SILVER, GOLD tiers get different prices)
- Override base price per product per customer segment

`DiscountTiers.jsx` (512 lines)

- Configures max discount per tier (BRONZE: 5%, SILVER: 10%, GOLD: 20%)
- Used by QuotationBuilder to enforce discount limits
- Sliders + input fields with real-time validation

`Warehouses.jsx` (577 lines)

- Visual stock grid: rows = products, columns = warehouses
- Inline stock adjustment with optimistic UI update
- Low stock alert (< 10 units turns red)

`UpsellRules.jsx` (504 lines)

- If-Then rule builder: IF product X is in cart → SUGGEST product Y
- Trigger types: ___CODESPAN_0_, _CODESPAN_1_, _CODESPAN_2___
- Rules feed the `getUpsellSuggestions` endpoint

`Users.jsx` (753 lines)

- Full user management: list, create, deactivate, reset password
- Role badge colors per role type
- Search/filter by role or status

---

16. UI Component Library

All in ___CODESPAN_0_**, exported via** _CODESPAN_1___.

StatusBadge.jsx

```
const CONFIG = {
  DRAFT:             { color: &#x27;#64748b&#x27;, bg: &#x27;rgba(100,116,139,0.15)&#x27;, label: &#x27;Draft&#x27; },
  PENDING_MANAGER:   { color: &#x27;#f59e0b&#x27;, bg: &#x27;rgba(245,158,11,0.15)&#x27;, label: &#x27;Pending Manager&#x27; },
  PENDING_FINANCE:   { color: &#x27;#f97316&#x27;, bg: &#x27;rgba(249,115,22,0.15)&#x27;, label: &#x27;Pending Finance&#x27; },
  APPROVED:          { color: &#x27;#10b981&#x27;, bg: &#x27;rgba(16,185,129,0.15)&#x27;, label: &#x27;Approved&#x27; },
  REJECTED:          { color: &#x27;#ef4444&#x27;, bg: &#x27;rgba(239,68,68,0.15)&#x27;, label: &#x27;Rejected&#x27; },
  SENT_TO_CUSTOMER:  { color: &#x27;#3b82f6&#x27;, bg: &#x27;rgba(59,130,246,0.15)&#x27;, label: &#x27;Sent to Customer&#x27; },
  CONFIRMED:         { color: &#x27;#10b981&#x27;, bg: &#x27;rgba(16,185,129,0.2)&#x27;, label: &#x27;Confirmed&#x27; },
  CANCELLED:         { color: &#x27;#64748b&#x27;, bg: &#x27;rgba(100,116,139,0.1)&#x27;, label: &#x27;Cancelled&#x27; },
};

export default function StatusBadge({ status, size = &#x27;sm&#x27; }) {
  const cfg = CONFIG[status] || CONFIG.DRAFT;
  return (
    &lt;span style={{
      color: cfg.color,
      background: cfg.bg,
      border: `1px solid ${cfg.color}40`,
      borderRadius: &#x27;9999px&#x27;,
      padding: size === &#x27;sm&#x27; ? &#x27;2px 10px&#x27; : &#x27;4px 14px&#x27;,
      fontSize: size === &#x27;sm&#x27; ? 11 : 13,
      fontWeight: 600,
    }}&gt;
      {cfg.label}
    &lt;/span&gt;
  );
}
```

KPICard.jsx

```
&lt;KPICard
  title=&quot;Confirmed Deals&quot;
  value={data.kpis.confirmedDeals}
  icon={CheckCircle}
  trend=&quot;+12% this month&quot;
  color=&quot;emerald&quot;
/&gt;
```

RiskScore.jsx

Circular progress ring showing risk 0–15 with color gradient: green → yellow → red.

Modal.jsx

Generic modal with backdrop, ESC key handler, focus trap.

NotificationDropdown.jsx

Bell icon with unread badge. Dropdown shows notification list. Mark-as-read on click.

---

17. LiveMarginBar — The Signature Component

File: `components/ui/LiveMarginBar.jsx` — **304 lines**

This is the most unique and impressive component. It's a **fixed bottom sticky bar** in QuotationBuilder that updates in real-time as the user adds/edits line items.

What it shows

```
■4,85,000     |  Margin: 34.2% [■■■■■■■■■■] 0% 15% 30% 50%+  |  ■ 5.2/15  |  ■ Auto-Approved  |  3 items
  LIVE TOTAL        GROSS MARGIN                                 RISK SCORE    APPROVAL PATH     LINE ITEMS
```

How it works

```
const metrics = useMemo(() =&gt; {
  for (const line of lines) {
    const effectivePrice = unitPrice * (1 - discount / 100);
    const lineRevenue = effectivePrice * qty;
    const lineCost = costPrice * qty;
    const lineTotal = lineRevenue * (1 + tax / 100);

    // Risk: discount-weighted per line
    const lineRisk = Math.min((discount / 20) * 10 + (discount &gt; 15 ? 5 : 0), 15);

    totalRevenue += lineRevenue;
    totalCost += lineCost;
    weightedRisk += lineRisk * lineRevenue;  // weighted by revenue
    totalWeight += lineRevenue;
  }

  const margin = (totalRevenue - totalCost) / totalRevenue * 100;
  const riskScore = weightedRisk / totalWeight;   // blended across lines
  return { total, margin, riskScore, discountToNextTier };
}, [lines]);
```

Approval Path Hint

```
if (avgDiscount &lt; 5)       → &#x27;■ Auto-Approved&#x27;
if (avgDiscount 5–10%)     → &#x27;■■ Needs Manager Review&#x27;
if (avgDiscount &gt; 10%)     → &#x27;■ Needs Manager + Finance&#x27;
```

Also shows: *"Add 2.3% more avg discount → triggers Finance Approval"*

Risk LED

```
&lt;div style={{
  background: riskColor,
  boxShadow: `0 0 16px ${riskColor}, 0 0 4px ${riskColor}`,  // glowing LED effect
  animation: riskScore &gt; 10 ? &#x27;ledpulse 1s infinite&#x27; : &#x27;none&#x27;,  // blinks red when critical
}} /&gt;
```

Responsive to Sidebar Width

```
const sidebarWidth = sidebarCollapsed ? 80 : 256;
// style: left: sidebarWidth  ← CSS transition 300ms to animate sidebar toggle
```

---

18. Utility Functions

`formatters.js` (48 lines)

```
export const formatCurrency = (val = 0) =&gt; {
  const n = Number(val || 0);
  if (n &gt;= 1e9) return `$${(n / 1e9).toFixed(1)}B`;
  if (n &gt;= 1e6) return `$${(n / 1e6).toFixed(1)}M`;
  if (n &gt;= 1e3) return `$${(n / 1e3).toFixed(0)}K`;
  return `$${n.toLocaleString()}`;
};

export const STAGES = [
  { id: &#x27;LEAD&#x27;,          label: &#x27;Lead Inflow&#x27;,   color: &#x27;border-slate-500&#x27; },
  { id: &#x27;QUALIFICATION&#x27;, label: &#x27;Qualification&#x27;, color: &#x27;border-blue-500&#x27; },
  { id: &#x27;DUE_DILIGENCE&#x27;, label: &#x27;Due Diligence&#x27;, color: &#x27;border-amber-500&#x27; },
  { id: &#x27;NEGOTIATION&#x27;,   label: &#x27;Negotiation&#x27;,   color: &#x27;border-purple-500&#x27; },
  { id: &#x27;CLOSED_WON&#x27;,    label: &#x27;Closed Won&#x27;,    color: &#x27;border-emerald-500&#x27; },
  { id: &#x27;CLOSED_LOST&#x27;,   label: &#x27;Closed Lost&#x27;,   color: &#x27;border-rose-500&#x27; },
];
```

INR formatting (used in QuotationBuilder and CustomerPortal):

```
const formatINR = (n) =&gt;
  new Intl.NumberFormat(&#x27;en-IN&#x27;, {
    style: &#x27;currency&#x27;, currency: &#x27;INR&#x27;, maximumFractionDigits: 0,
  }).format(n ?? 0);
// Output: ■4,85,000
```

---

19. Routing & Role-Based Access Control

Route Table

URL	Component	Auth Required	Roles
`/login`	Login	No	—
`/signup`	Signup	No	—
`/portal/:token`	CustomerPortal	No	magic token
`/portal/login`	PortalLogin	No	—
`/dashboard`	Dashboard	Yes	All
`/products`	ProductsPage	Yes	ADMIN, SALES_MANAGER
`/price-lists`	PriceListsPage	Yes	ADMIN, SALES_MANAGER
`/discount-tiers`	DiscountTiersPage	Yes	ADMIN, SALES_MANAGER
`/warehouses`	WarehousesPage	Yes	ADMIN, SALES_MANAGER, FINANCE
`/subscription-plans`	SubscriptionPlansPage	Yes	ADMIN, SALES_MANAGER
`/upsell-rules`	UpsellRulesPage	Yes	ADMIN, SALES_MANAGER
`/users`	UsersPage	Yes	ADMIN
`/quotations`	QuotationsList	Yes	All
`/quotations/new`	QuotationBuilder	Yes	SALES_REP, SALES_MANAGER, ADMIN
`/quotations/:id`	QuotationBuilder	Yes	All
`/pipeline`	PipelineKanban	Yes	All
`/approvals`	ApprovalQueue	Yes	SALES_MANAGER, FINANCE, ADMIN
`/fulfillment`	FulfillmentPage	Yes	All
`/subscriptions`	SubscriptionsPage	Yes	All
`/invoices`	InvoicesPage	Yes	All
`*`	redirect → /dashboard	—	—

Guard HOC (in App.jsx)

```
function Guard({ children, roles }) {
  const { user } = useAuthStore();
  if (!user) return &lt;Navigate to=&quot;/login&quot; replace /&gt;;    // not logged in
  if (roles &amp;&amp; !roles.includes(user.role))
    return &lt;Navigate to=&quot;/dashboard&quot; replace /&gt;;          // wrong role
  return children;                                          // allowed
}
```

`replace` means the redirect doesn't add to browser history — user can't click Back to the protected page.

---

20. Common Interview Q&A

Q: Why React 18? A: Concurrent features, automatic batching of state updates, and ___CODESPAN_0_ **support. We use** _CODESPAN_1_ **(not** _CODESPAN_2___).

Q: Why Zustand over Redux / Context? A: Zero boilerplate, tiny bundle (~1KB), works outside React components (critical for Axios interceptors via ___CODESPAN_0_**), and** _CODESPAN_1___ middleware gives free localStorage sync.

Q: How does the access token stay fresh? A: Two-layer strategy. ___CODESPAN_0_ **refreshes on page load. Axios response interceptor auto-retries on 401 with a silent refresh call using the httpOnly cookie. The** _CODESPAN_1___ flag prevents loops.

Q: Why not store token in localStorage? A: The *access token* is in Zustand (memory + localStorage for hydration). The *refresh token* is in an httpOnly cookie — inaccessible to JavaScript, safe from XSS.

Q: How does LiveMarginBar work without a re-render on every keystroke? A: It uses ___CODESPAN_0_ **— React only recomputes when the** _CODESPAN_1_ **array changes. The bar itself is a pure functional component with no internal state — it's driven entirely by the parent's** _CODESPAN_2___ prop.

Q: How is Socket.io connected per page? A: Each page that needs real-time creates its own ___CODESPAN_0_ **connection in a** _CODESPAN_1_ **with a cleanup** _CODESPAN_2_ **on unmount. The server manages rooms — clients join specific rooms like** _CODESPAN_3___ so messages are scoped.

Q: How does the Customer Portal work without login? A: The URL itself is the credential — `/portal/:token` where token is a UUID. The backend validates the token and returns the quotation. No JWT needed for portal access.

Q: What happens when a quotation PDF is downloaded? A: Axios calls ___CODESPAN_0_ **with** _CODESPAN_1_**. The server streams a** _CODESPAN_2_ **generated PDF. Frontend creates a blob URL, clicks a virtual** _CODESPAN_3___ tag programmatically,

then revokes the URL.

Q: How does drag-and-drop in PipelineKanban work? A: Uses ___CODESPAN_0_**.** _CODESPAN_1_ **receives source column and destination column. It calls** _CODESPAN_2___ to persist, and optimistically updates local state immediately for snappy UX.

Q: What is the blended risk score formula? A: ___CODESPAN_0_ **— revenue-weighted average of per-line risk. Per-line risk =** _CODESPAN_1___. Score range 0–15.

Q: How are upsell suggestions generated? A: Frontend sends current product IDs to ___CODESPAN_0_**. Backend queries** _CODESPAN_1_ **table where** _CODESPAN_2_ **matches any cart product. Returns** _CODESPAN_3_ **records with** _CODESPAN_4___ text.

Quick Cheatsheet for Demo

```
Login page:       http://localhost:5173/login
Dashboard:        http://localhost:5173/dashboard
New Quotation:    http://localhost:5173/quotations/new
Approval Queue:   http://localhost:5173/approvals
Customer Portal:  http://localhost:5173/portal/portal-token-acme-004
Products Admin:   http://localhost:5173/products
Kanban Pipeline:  http://localhost:5173/pipeline
Fulfillment:      http://localhost:5173/fulfillment
```

Demo accounts (click cards on login page):

Role	Email	Password	Sees
Admin	admin@dealflow.com	Admin@123	Everything
Sales Rep	priya@dealflow.com	Rep@123	Create quotations
Manager	manager@dealflow.com	Manager@123	Approve stage 1
Finance	finance@dealflow.com	Finance@123	Approve stage 2
Customer	buyer@acme.com	Customer@123	Customer portal

Generated: 2026-09-05 | DealFlow360 Frontend Reference v1.0